

Elastic pro Go developera a migrace z Logstash na Beats a Elastic Agent

Cíl dokumentu

Tento dokument slouží jako podrobný průvodce pro Go developera, který chce:

- Rozumět tomu, co Elastic Stack prakticky je a jaké komponenty se ho týkají v běžné backend / mikroslužbové architektuře.
- Používat Elasticsearch z Go aplikací pomocí oficiálního Go klienta.
- Navrhovat indexy, dokumenty a dotazy tak, aby byly dobře škálovatelné a udržitelné.
- Pochopit roli Logstash, Beats a Elastic Agentu v ingest vrstvě.
- Naplánovat a realizovat přechod z Logstash na Beats a zejména na Elastic Agent.

Dokument je psán prakticky z pohledu vývojáře, který o Elasticu ví základní věci (co je index, dokument), ale chce se dostat na úroveň, kdy umí navrhovat integraci a rozumí důsledkům rozhodnutí v produkci.

1. Elastic Stack – rychlý, ale praktický přehled

Elastic Stack typicky zahrnuje:

- **Elasticsearch** – distribuovaný dokumentový datastore a vyhledávač nad REST/HTTP API.
- **Kibana** – webové UI pro vizualizaci dat, správu indexů, observabilitu a bezpečnost.
- **Ingest vrstvu** – nástroje pro sběr dat (Logstash, Beats, Elastic Agent, případně vlastní aplikace).
- **Další komponenty** – například APM, Security řešení, Fleet a integrace pro cloudové platformy.

Z pohledu Go developera jsou klíčové dvě linie:

1. **Aplikační integrace** – tedy jak z Go služby zapisovat a číst data z Elasticsearch.
2. **Ingest infrastruktura** – jak se logy, metriky a další signály dostanou z běhového prostředí a infrastruktury do Elasticu.

1.1 Elasticsearch jako služba

Elasticsearch typicky běží buď:

- jako self-managed cluster (on-prem nebo v cloudu),
- nebo jako managed služba (Elastic Cloud, případně integrace do cloudu třetí strany).

Pro Go aplikaci je důležité, že Elasticsearch je vždy **externí síťová služba** s určitým SLA, latencí a bezpečnostní konfigurací (TLS, API tokeny, basic auth, OIDC, ...).

Z toho vyplývá, že se musí řešit:

- time-outy a retry logika,
- "backpressure" – co se stane, když Elasticsearch nestíhá,
- monitoring a metriky klienta,
- správa schémat a indexů (migrační strategie).

2. Oficiální Go klient pro Elasticsearch

Elastic poskytuje oficiální Go klient `go-elasticsearch`, který je doporučený způsob, jak z Go aplikace mluvit na Elasticsearch API.

2.1 Instalace a verze

Klient je verzovaný podle hlavní verze Elasticsearch (např. v8).

Typicky ho přidáš do projektu přes `go get` s konkrétní hlavní verzí.

Důležité body z hlediska verzování:

- Vyplatí se sladit hlavní verzi klienta s hlavní verzí Elasticsearch clusteru.
- Menší minor verze klienta obvykle podporují více minor verzí serveru, ale je dobré držet je relativně aktuální.
- Při upgradu Elasticsearch kontroluj `breaking changes` i v klientovi (typed API se může mírně měnit).

2.2 Základní inicializace klienta

Klienta typicky inicializuješ jednou při startu služby a předáš ho ve formě dependency do částí, které pracují s daty.

Klíčové body konfigurace:

- URL na cluster (nebo koordinující node/load balancer).
- Autentizace – nejčastěji API token nebo basic auth, případně cloud ID.
- TLS nastavení – validace certifikátu, custom CA, případně mTLS.
- Nastavení time-outů a transportu (např. `custom http.Client`).

Dobrá praxe:

- Zabalit klienta do interního balíčku, který vystaví jen potřebné metody (například `SearchOrders`, `IndexLog`, ...) místo toho, aby zbytek kódu pracoval přímo s nízkoúrovňovým API.
- Doplnit metriky (latence, počet requestů, chybovost, počet retry pokusů) a logování failed requestů.

2.3 Typed API a esdsl

Moderní přístup doporučený Elasticem je používat **typed API** společně s buildery `esdsl`.

Výhody typed API:

- Silnější typová kontrola nad dotazy – místo ručního skládání JSONu.
- Lepší refaktoring – změna názvu pole nebo struktury se projeví v kompilaci.
- Menší riziko chyb v JSON struktuře, které by se jinak projevily až v runtime.

Pro Go developera to znamená:

- Definovat si struktury pro requesty a odpovědi.
- Sestavovat dotazy a agregace pomocí builderů.
- Používat generické funkce a helpery pro opakující se patterny (např. stránkování, filtrování podle tenant ID, index aliasy).

2.4 Bulk API

Bulk API je klíčové pro efektivní zápis většího množství dokumentů z Go aplikace.

Doporučení:

- Používat buffer, který shromažďuje dokumenty a periodicky nebo po dosažení velikosti provádí bulk request.
- Řešit retry jen u některých chyb (např. 429 – too many requests, dočasné network chyby) a dávat si pozor na idempotenci (např. používat deterministické `_id`).
- Mít metriku na úspěšnost a latenci bulk operací.

Vzory chování:

- Oddělit kód pro konverzi interního modelu na dokument do Elasticsearch od samotného volání bulk API.
- Validovat velikost batch (počet dokumentů, velikost payloadu v bajtech).

3. Návrh indexů a dokumentů z pohledu Go aplikace

Do Elasticsearch nejčastěji posíláš:

- logy,
- metriky a APM data,
- doménové dokumenty (produkty, uživatelské akce, objednávky, ...),
- bezpečnostní eventy (SIEM případy).

3.1 Index jako "tabulka" vs. realita

Elasticsearch index se často vnímá jako tabulka, ale realita je jiná:

- Index je spíše "partitionovaný" store – má shardy, replika shardy a lifecycle management.
- Každé pole má typ a analyzátor – například text vs. keyword, date, long, double, boolean.
- Změna typů polí je mnohem náročnější než v klasické databázi – často vyžaduje nový index a reindexing.

Pro Go vývojáře to znamená, že musí existovat nějaký schématický kontrakt mezi aplikací a indexem:

- struktura Go structu odpovídá mappingu v indexu,
- evoluce schématu (přidání pole, změna semantics) je řízená – typicky přes migrační skripty,
- v kódu musí být verzování indexů nebo aliasů (např. logs-v1, logs-v2 a alias logs-current).

3.2 Mapping a analyzátor

Základní rozhodnutí, která má dopad na dotazování i výkon:

- Pole typu text jsou vhodná pro fulltext, jsou analyzovaná a dělí se na tokeny.
- Pole typu keyword jsou vhodná pro přesné filtrování, agregace a sort.
- date pole potřebuje jasný formát.
- Nested objekty vs. flat struktura – pro některé use-case je lepší data denormalizovat.

Go aplikace by měla mít jasný model, který odpovídá těmto typům – například pomocí struct tagů pro JSON serializaci.

3.3 Index lifecycle a time-based indexy

Pro logy a metriky je běžné používat time-based indexy:

- Např. jeden index na den nebo měsíc.
- Správa přes ILM (Index Lifecycle Management) – rollover, shrink, delete.

Důsledky pro Go aplikaci:

- Zapisuješ typicky přes alias (např. logs-app), který ukazuje na aktuální write index.
- Aplikace by neměla muset znát konkrétní suffixy indexů, to je úkol správy clusteru.
- Při čtení můžeš dotazovat alias nebo konkrétní pattern (např. logs-app-*).

4. Dotazování a agregace z Go

4.1 Základní patterny dotazů

Běžné dotazovací patterny:

- fulltext vyhledávání (match, multi_match),
- filtrace podle polí (term, range),
- kombinace bool dotazů (must, should, filter, must_not),
- časové filtry přes @timestamp.

Go klient by měl mít helpery pro opakující se části:

- přidání tenant_id filtru,
- nastavení size, from a sortu,
- přidání agregací pro potřeby dashboardů.

4.2 Agregace

Agregace jsou extrémně silná část Elasticsearch:

- terms agregace pro rozdělení podle kategorií,
- date_histogram pro časové řady,
- metrics agregace (avg, sum, max, percentiles, ...),
- pipeline agregace pro odvozování hodnot.

Z Go typicky děláš:

- helper funkce, které sestaví konkrétní agregaci pro daný use-case,
- mapování výsledků na vlastní datové struktury pro UI / API.

Důležité je **nedělat příliš obecné funkce**, které vrácí "mapu map", ale mít jasné typy, aby se lépe refaktorovalo.

4.3 Stránkování a deep pagination

Elasticsearch není klasická databáze pro deep pagination (např. stránka 10000 po 10 položkách).

Možnosti:

- klasické from/size pro menší offsety,
- search_after pro stránkování podle hodnot určitého sort klíče,
- použití scroll jen pro batch processing, ne pro interaktivní dotazy.

Go aplikace by měla skrývat detaily v API vrstvě – klient např. dostává cursor místo offset.

5. Ingest vrstva: Logstash, Beats a Elastic Agent

Z pohledu datového toku:

- Zdroje dat → ingest (Logstash / Beats / Elastic Agent / vlastní aplikace) → Elasticsearch.

5.1 Logstash – silná, ale těžší vrstva

Logstash je plnohodnotný ingest engine, který nabízí:

- mnoho vstupních pluginů (TCP, UDP, Kafka, Beats, ...),
- filtry pro transformaci dat (grok, mutate, date, json, ...),
- výstupy do různých systémů (Elasticsearch, S3, další brokery, ...).

Hodí se tam, kde potřebuješ:

- složitější parse logů (např. grok z textových logů),
- enrichment z externích zdrojů (GeoIP, databáze),
- komplexní routing (větvení pipeline, fan-out do více cílů),
- integraci se staršími systémy.

Nevýhody:

- vyšší nároky na zdroje (Java, paměť),
- složitější provoz – správa pipeline, nasazování konfigurací, škálování,
- riziko "snowflake" konfigurací, když se neudrží konzistentní styl.

5.2 Beats – lehčí specialisté

Beats jsou lehké agenty, každý specializovaný na konkrétní typ dat:

- Filebeat – logy ze souborů,
- Metricbeat – metriky,
- Packetbeat – síťový traffic,
- Winlogbeat – Windows logy,
- Auditbeat – auditní eventy.

Výhody:

- nízký footprint,
- jednoduchá konfigurace pro běžné use-case,
- přímočará integrace s Elasticsearch.

Nevýhody:

- každý Beat je samostatná binárka a konfigurace,
- správa mnoha agentů může být náročná (verze, konfigurace),

- složitější scénáře stále často vyžadují Logstash.

5.3 Elastic Agent – sjednocený přístup

Elastic Agent vznikl jako odpověď na fragmentaci kolem Beats.

Klíčové vlastnosti:

- jedna binárka pro sběr logů, metrik, APM, bezpečnostních dat a dalšího,
- centrální správa přes Fleet v Kibana (politiky, integrace, upgrade),
- integrace pro velké množství běžných technologií (Docker, Kubernetes, Nginx, databáze, cloud provider logy, ...).

Výhody proti Beats:

- jednotná správa konfigurací a verzí,
- jednodušší onboarding nových hostů a služeb,
- lepší pozice do budoucna – Elastic investuje hlavně do Agentu a Fleet.

Z pohledu Go developera:

- pro aplikační logy běžících na hostu nebo v Kubernetesu je často nejjednodušší nasadit Elastic Agent (např. jako DaemonSet),
- v aplikaci pak stačí logovat do souboru / stdout ve strukturovaném formátu, který Agent umí číst,
- transformace (parsování, obohacení) může dělat ingest pipeline nebo samotná integrace v Elastic Agentu.

6. Přechod z Logstash na Beats a Elastic Agent – koncepční rámec

Migrace z Logstash na lehčí ingest (Beats/Elastic Agent) se dá chápat jako postupné "odhazování" zbytečné complexity.

6.1 Zmapuj, co Logstash dělá

Než začneš nahrazovat Logstash, potřebuješ jasný obrázek:

- Jaké **inputs** používáš? (např. Beats, syslog, Kafka, TCP)
- Jaké **filtry** a transformace probíhají? (grok, json, mutate, geoip, translate, ruby skripty)
- Jaké **outputs** existují? (Elasticsearch, S3, další logovací systémy)
- Jaké jsou **závislosti**? (např. externí databáze pro enrichment, DNS, služby pro lookup)

Cílem je rozdělit Logstash funkce do kategorií:

- čistý transport (příjem a posílání dál bez větší změny),
- jednoduché transformace (parse JSON, rename field, přidání statického pole),
- složité transformace a enrichment,

- routing/fan-out.

6.2 Co lze přesunout do ingest pipeline

Elasticsearch má vlastní ingest pipeline, které umí část práce Logstash nahradit:

- procesory pro parse (grok, json, csv, ...),
- enrichment (geoip, user_agent),
- mutate (rename, remove, set),
- podmínky (if logika) nad poli.

Výhody přesunu do ingest pipeline:

- méně komponent v infrastruktuře,
- transformace jsou verzované a uložené přímo v clusteru,
- jednodušší škálování – ingest se škáluje spolu s datovými nody.

Nevýhody:

- složitější transformace nebo integrace s externími systémy se mohou dělat hůř,
- část logiky se "ztratí" z klasického DevOps toolchainu, pokud tým není zvyklý spravovat ingest pipeline jako kód.

6.3 Co lze přesunout do Beats / Elastic Agent

Beats a Elastic Agent nabízejí vestavěné moduly / integrace, které umí:

- číst logy z konkrétních cest nebo z container stdout,
- přidávat metadata (kubernetes, cloud, host, service),
- posílat data přímo do Elasticsearch nebo přes Logstash,
- v některých případech provést základní parsing.

To znamená, že část konfigurace, kterou dnes řešíš v Logstash (např. rozpoznání log formátu konkrétního software), může být přenesena do integrace v Elastic Agentu.

7. Praktický migrační plán krok za krokem

Níže je jeden z rozumných scénářů, jak přejít z Logstash na Beats/Elastic Agent.

Krok 1: Inventarizace a dokumentace stávajícího stavu

- Seznam všech Logstash pipeline (vstupy, filtry, výstupy).
- U každé pipeline anotuj:
 - typ dat (logy, metriky, bezpečnostní eventy),
 - zdroj (konkrétní aplikace, hosty, kontejnery),
 - cílové indexy a patterny,

- specifiká (např. složitý grok, dependency na externí systém).

Výstupem je mapa, která ti umožní rozhodovat po částech, ne vše naráz.

Krok 2: Identifikace "easy wins"

Najdi pipeline, které:

- mají Logstash hlavně jako transport (např. Filebeat → Logstash → Elasticsearch bez většího filtru),
- nebo používají jednoduché transformace (parse JSON, rename field), které lze snadno přepsat do ingest pipeline nebo do integrace Elastic Agentu.

Tyto pipeline jsou vhodné jako první kandidáti na migraci.

Krok 3: Návrh cílové architektury

Rozhodni se pro každý typ dat:

- Bude zdroj dat čist Elastic Agent (např. DaemonSet na Kubernetesu, agent na hostu)?
- Použije se některý existující modul / integrace, nebo bude potřeba custom log parser?
- Půjde data posílat přímo do Elasticsearch (s ingest pipeline), nebo zůstane v cestě Logstash například pro speciální routing?

Pro většinu standardních logů a metrik je cílový stav:

- Elastic Agent (nebo Beats) → Elasticsearch ingest pipeline.

Logstash pak zůstává jen pro specializované scénáře.

Krok 4: Implementace první migrační větve

Na jednom typu logů (např. aplikace X):

- Nasadíš Elastic Agent / Beats na cílové hosty / kontejnery.
- Zkonfiguruješ ingest pipeline nebo integraci tak, aby produkovala stejná pole jako předchozí Logstash pipeline.
- Ponecháš dočasně oba toky:
 - původní (Logstash),
 - nový (Elastic Agent → ingest pipeline) do jiného indexu.

Krok 5: Paritní testování

- Pro vybrané časové období porovnáš dokumenty z obou toků:
 - strukturu polí,
 - datové typy,
 - klíčové hodnoty (např. `service.name`, `log.level`, `@timestamp`).
- Zkontroluješ dopad na:

- latenci ingestu,
- objem dat (velikost indexu),
- chybovost a retry.

Pokud se zjistí rozdíl, upravíš ingest pipeline nebo konfiguraci agenta.

Krok 6: Postupné přepnutí provozu

Jakmile je parita dostatečná:

- V Kibana dashboardech a alertingu přepneš dotazy z původních indexů na nové.
- U nového toku nastavíš aliasy na stejné jméno, jaké používal starý index (pokud je to vhodné).
- Vypneš starý tok přes Logstash pro daný typ dat.

Tento proces opakuješ po částech, dokud většina běžných pipeline neběží přes Elastic Agent/Beats.

Krok 7: Konsolidace a úklid

Po migraci hlavních toků:

- Odstraníš nepoužívané Logstash konfigurace.
- Zredukuješ velikost clusteru Logstash (pokud už není tolik zatížený).
- Zdokumentuješ finální architekturu a procesy nasazování konfigurací agenta a ingest pipeline.

8. Dopady na Go aplikace a tým

Migrace ingest vrstvy se dotkne i Go vývojářů, i když aplikace samotná může zůstat beze změny.

8.1 Logovací formát a struktura

- Pokud aplikační logy čte Elastic Agent přímo jako soubor nebo stdout, dává smysl sjednotit logovací formát.
- Doporučení je používat strukturované logování (JSON) – např. prostřednictvím knihoven jako zap, logrus s JSON formátorem nebo standardní log balík zabalený do vlastního writeru.
- Důležité je:
 - mít konzistentní pole pro čas (@timestamp nebo time),
 - service.name, service.version, případně environment,
 - úroveň logu (log.level),
 - trace / span ID, pokud používáte distributed tracing.

Elastic Agent / ingest pipeline pak vytáhne tato pole a naváže je na standardizované ECS (Elastic Common Schema) pole, což výrazně usnadní práci v Kibana.

8.2 Observabilita ingestu

Pro tým Go vývojářů je důležité mít přehled, co se děje s jejich logy a daty:

- Dashboardy pro ingest pipeline (počet dokumentů, chybové rate, latence).
- Alerty, pokud data z konkrétní služby přestanou proudit.
- Logy Elastic Agentu a ingest pipeline přístupné přes Kibana, aby se daly rychle řešit problémy.

8.3 Border případy, kdy Logstash zůstává

Některé případy mohou i po migraci vyžadovat Logstash:

- komplikované obohacení z externího systému,
- integrace s legacy protokoly,
- složitý routing do více cílových systémů.

V takových situacích je dobré Logstash brát jako specializovaný nástroj, ne jako výchozí volbu pro všechny logy.

9. Best practices a anti-patterny

9.1 Best practices

- Používej oficiální Go klient a typed API – nepiš si vlastní generování JSON dotazů všude po kódu.
- Zabal přístup k Elasticsearch do interní vrstvy / balíčku a zbytku kódu exportuj doménově smysluplné metody.
- Logy generuj ve strukturovaném formátu, který se dobře mapuje na ECS.
- Validuj schémata indexů a udržuj je jako kód (např. JSON/YAML v repozitáři + migrační skripty).
- U ingest pipeline stejně jako u kódu aplikace dělej code review a testování.

9.2 Anti-patterny

- Přímé připojování všech částí kódu na nízkoúrovňového klienta bez abstrahování.
- Volání Elasticsearch bez time-outu, bez retry a bez metrik.
- Logstash pipeline, které dělají "všechno" – složitý monolit ingestu, který se těžko mění.
- Kopírování konfigurací Elastic Agentu / Beats bez pochopení, co dělají.

10. Jak začít prakticky – doporučený postup pro Go tým

1. Zaveď oficiální Go klient a vytvoř tenkou abstrakční vrstvu, která skrývá nízkoúrovňové API.
2. Vytvoř základní index a mapping pro jeden klíčový use-case (např. hledání objednávek) a implementuj první sadu dotazů.
3. Standardizuj logování v Go službách, aby logy byly konzistentní a strukturované.
4. Nasadíš Elastic Agent (např. v Kubernetesu) pro sběr logů a základních metrik.
5. Začni s malou, dobře definovanou migrací z Logstash na Elastic Agent pro jednu službu / typ logů.
6. Sleduj metriky a přizpůsob konfiguraci ingest pipeline.
7. Postupně rozšiřuj migraci, dokud se Logstash nepoužívá jen tam, kde je to skutečně nutné.

Tento přístup ti umožní postupně zvyšovat úroveň zralosti práce s Elasticem, aniž bys musel dělat velký "big bang" redesign.